



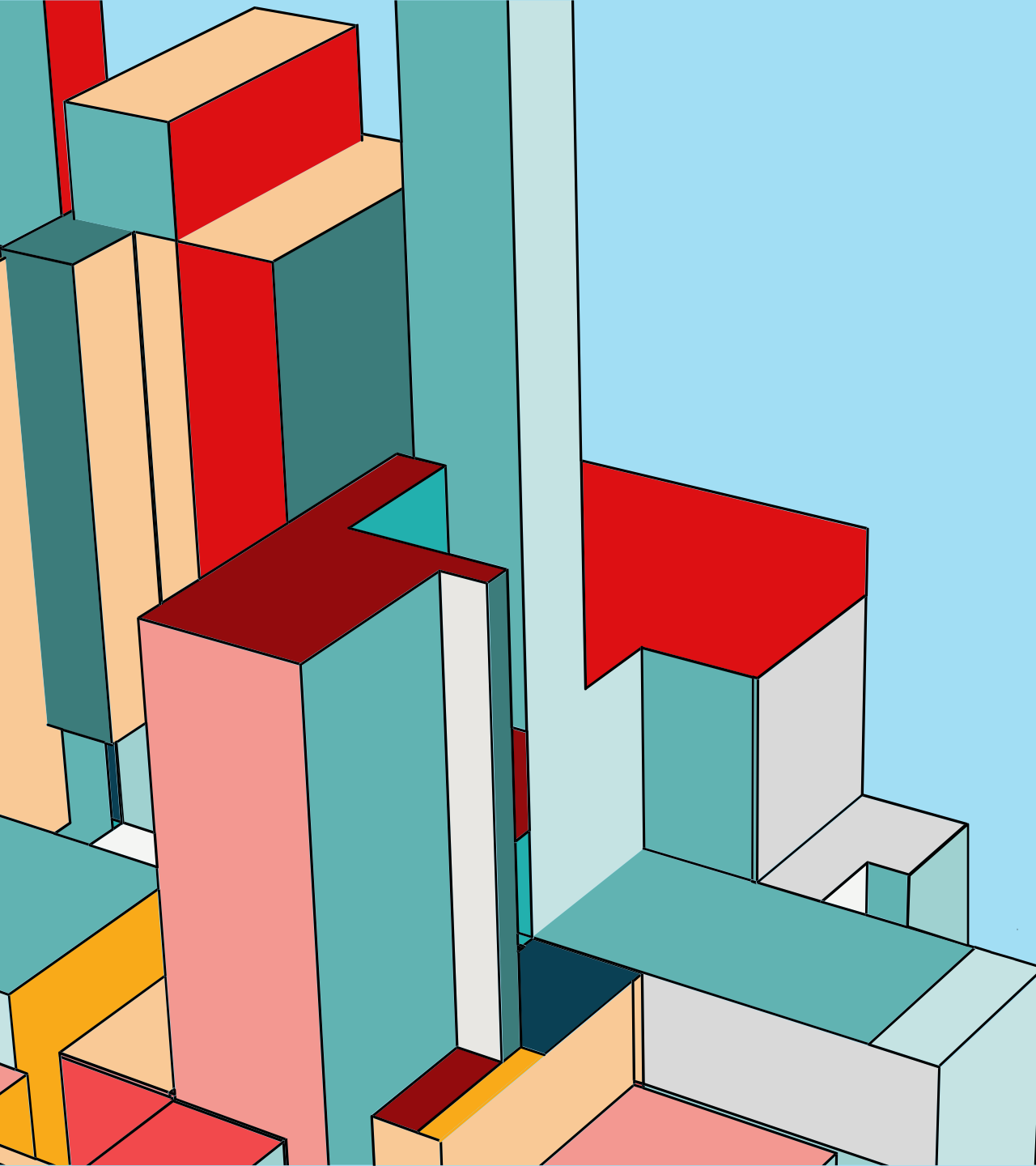
Interest form



OPTRPI Discord

- OPTRPI is a programming competition run by RCPC and UPE
- We have two tracks so everyone can join regardless of experience :D
 - Pre- Algo: for students who haven't taken an algorithms course
 - Post-Algo: for students who are in Algo or have taken Algo
- Join RCPC if you want to train for the upcoming competition!
 - Also practice for any technical interviews if you want a job
 - Wednesdays from 6:00-8:00 PM in Sage 5101
- This year's competition is on April 4 from noon-6pm in Lally.



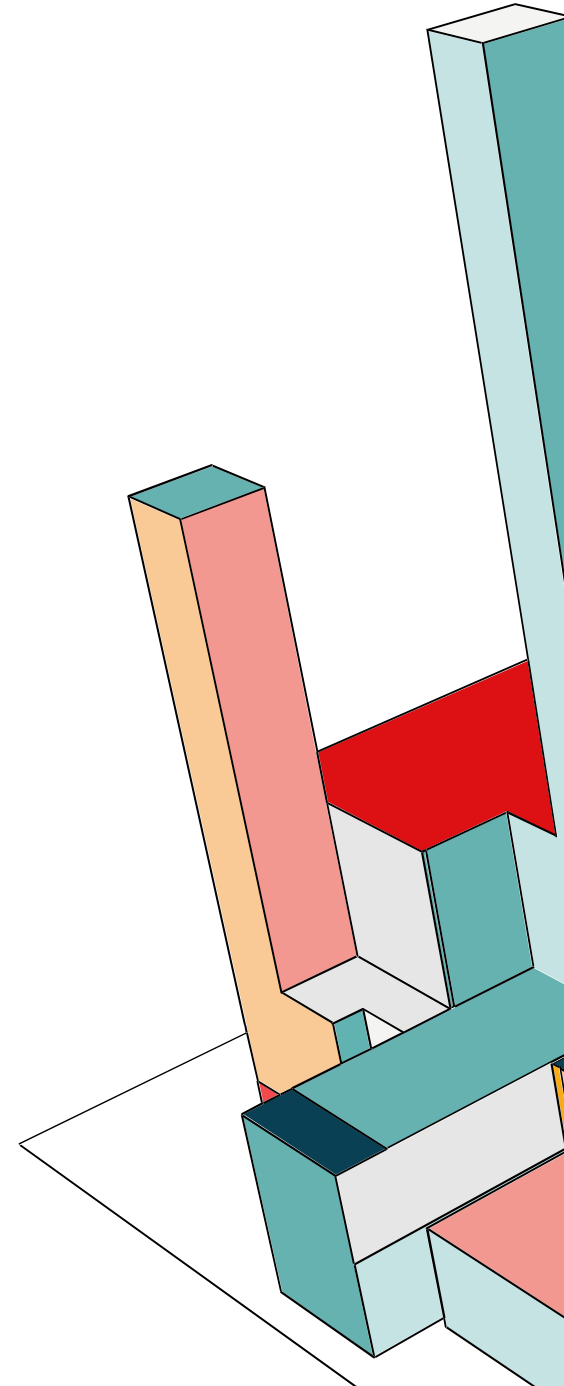


LECTURE 16 **DYNAMIC PROGRAMMING**

CSIS 2300 Introduction to
Algorithms, Spring 2026
Dr. Dan DiTursi

ANNOUNCEMENTS & REMINDERS

- No HW currently due; no lab next week
- Programming contest announcement



FIBONACCI NUMBERS & RECURSION

One of the very first examples used to teach the concept of recursion is always the Fibonacci sequence:

1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ...

The sequence is naturally defined recursively as $F_n = F_{n-1} + F_{n-2}$, with base cases of $F_1 = 1$ and $F_2 = 1$.

```
def fib(n):  
    if n < 3: # cheating a bit here, but ~(ツ)~  
        return 1  
    return fib(n-1) + fib(n-2)
```

SUBMITTY POLL #1

```
def fib(n):  
    if n < 3:  
        return 1  
    return fib(n-1) + fib(n-2)
```

What is the asymptotic runtime of this code?



FIBONACCI NUMBERS THE SMART WAY

Of course, we know that this is not the best way to do this.

Even though the best intuitive definition of the Fibonacci numbers is recursive, the best algorithm for producing them involves a space-for-time tradeoff.

```
def fib(n):  
    F = empty array of size n+1  
    F[1] = 1; F[2] = 1  
    for i = 3 to n:  
        F[i] = F[i-1] + F[i-2]
```

This produces the first n Fibonacci numbers in $O(n)$ time.

CONGRATULATIONS! YOU NOW UNDERSTAND DYNAMIC PROGRAMMING!

```
def fib(n):  
    F = empty array of size n+1  
    F[1] = 1; F[2] = 1  
    for i = 3 to n:  
        F[i] = F[i-1] + F[i-2]
```

The key idea here is that the same recursive calls in the first **fib** get repeated over and over again.

If we were doing it by hand, we would never recalculate that many times – we would write the answers down!

Doing so in the second **fib** speeds things up substantially.

OVERVIEW OF DYNAMIC PROGRAMMING

The core feature of dynamic programming is *memoization* – recording the answers to subproblems.

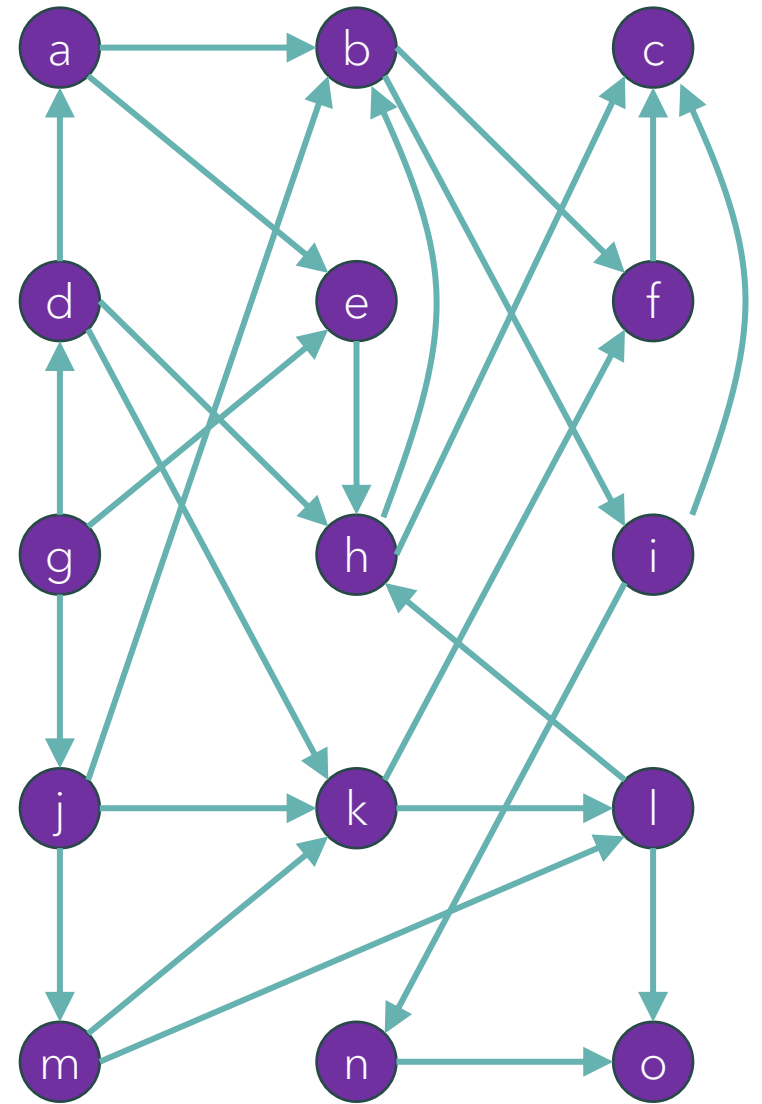
We find dynamic programming most useful when:

- The answer to a larger problem recursively depends upon the answers to subproblems. (Just as in divide-and-conquer)
- The subproblems overlap: Multiple subproblems depend upon the same subsubproblem. (Unlike D&C)

DAGS & SHORTEST PATH

Recall the concept of a *directed acyclic graph* or *DAG*.

Let's take this DAG and add weights onto the edges.



DAGS & SHORTEST PATH

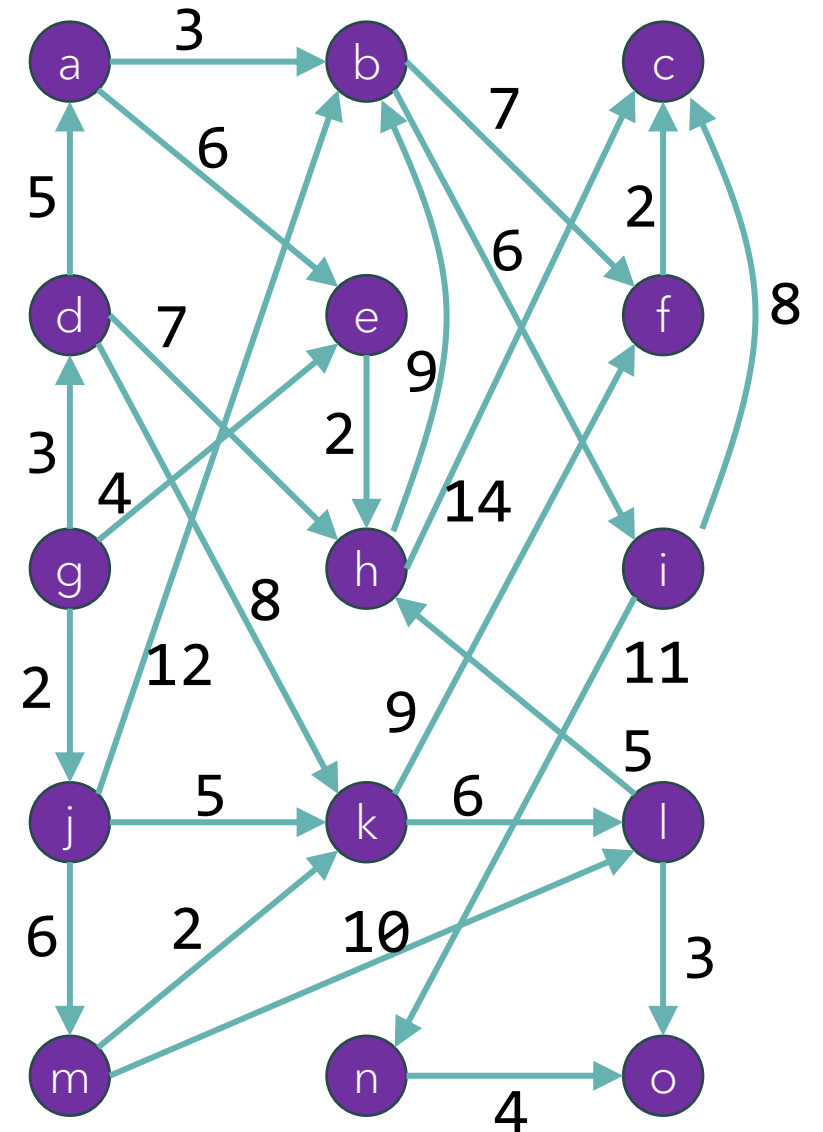
Recall the concept of a *directed acyclic graph* or *DAG*.

Let's take this DAG and add weights onto the edges.

Now, how would we find the shortest (lowest-weight) path from, say, *g* to *c*?

Obviously, we could use Dijkstra...

... but does the structure of the input (a DAG) lend itself to a better solution?



DAGS & SHORTEST PATH

Consider our target node c .

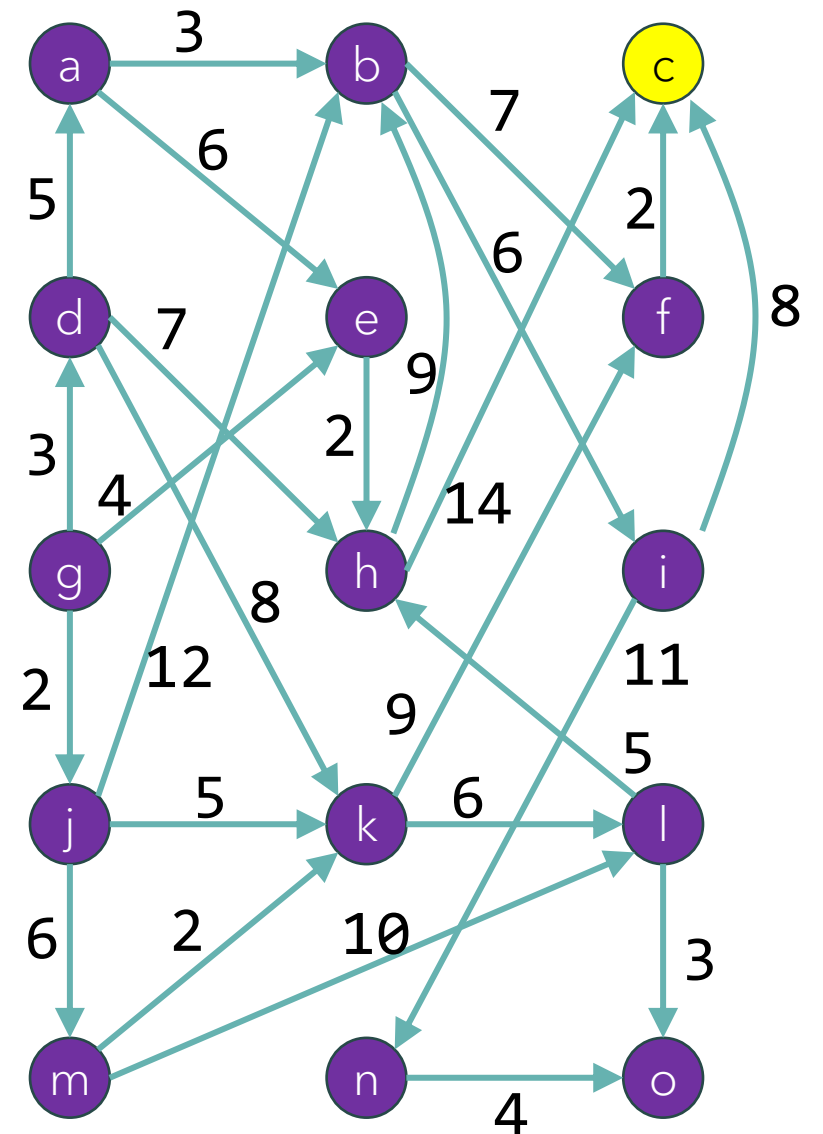
What can we say about the shortest path that leads to c ?

The second-to-last node is f , h , or i .

So we can say that $dist(g, c) = \min(2 + dist(g, f), 14 + dist(g, h), 8 + dist(g, i))$

And $dist(g, f) = \min(7 + dist(g, b), 9 + dist(g, k))$

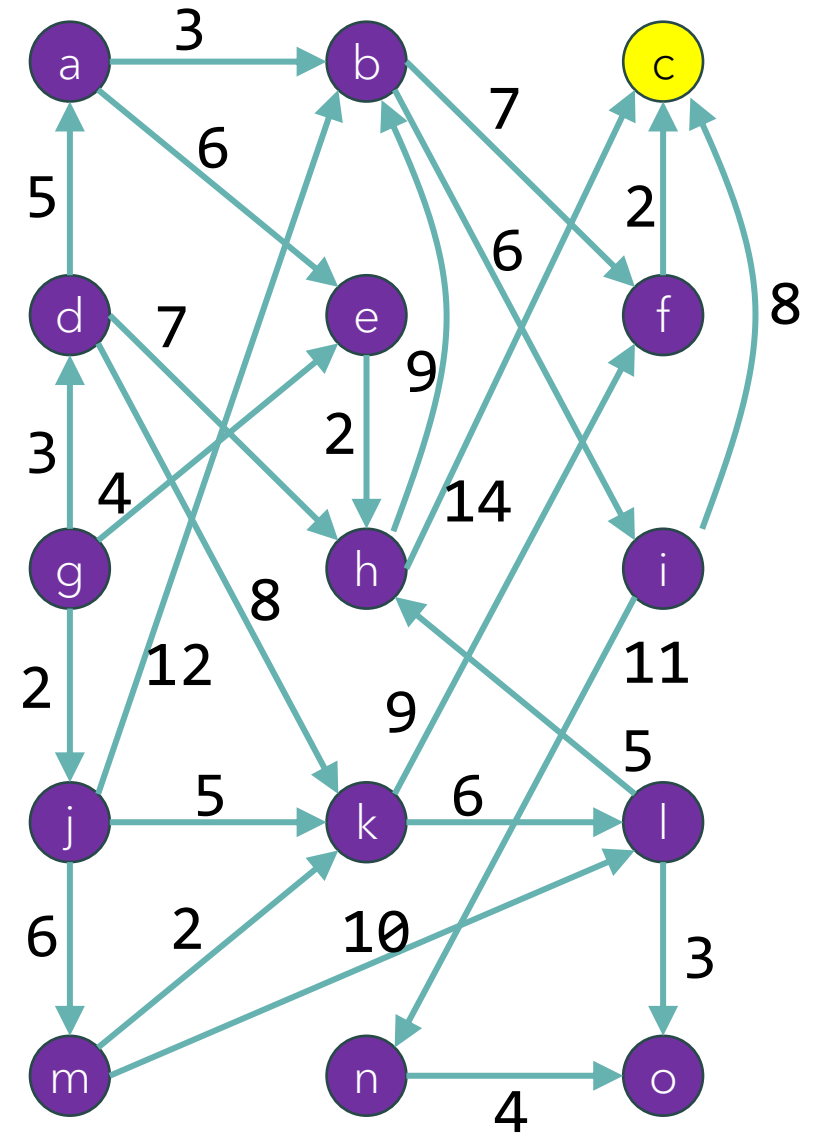
If only there was a way to list the nodes in order along the path, we could pull the same trick as we did for Fibonacci...



TOPOLOGICAL SORT

Remember topological sort?

We can use depth-first search to order the nodes in a DAG so that later nodes in the list never have edges back to earlier ones.



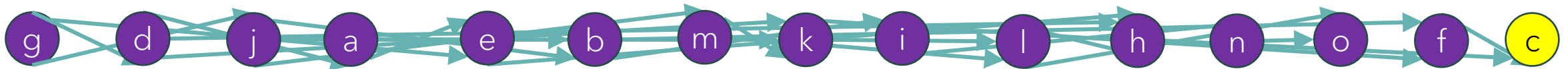
TOPOLOGICAL SORT

Remember topological sort?

We can use depth-first search to order the nodes in a DAG so that later nodes in the list never have edges back to earlier ones.

Once we have done that, we just calculate distances from the source in order from left to right – each one will only depend upon distances to its left.

Since DFS is $O(V + E)$, this is faster than Dijkstra's algorithm.



DAG SHORTEST PATH VIA TOPOLOGICAL SORT

```
# s = source vertex for path; t = target vertex for path
def short(dag,s,t):
    V = topo_sort(dag)
    for v in V:
        dist(v) =  $\infty$ 
    dist(s) = 0
    for v in V from s.next() to t: # walk down sorted vertices
        for each incoming edge to v (u,v):
            if dist(u) + weight(u,v) < dist(v):
                dist(v) = dist(u) + weight(u,v)
    return dist(t)
```



DYNAMIC PROGRAMMING: RECURSIVE STRUCTURE, ITERATIVE ALGORITHM

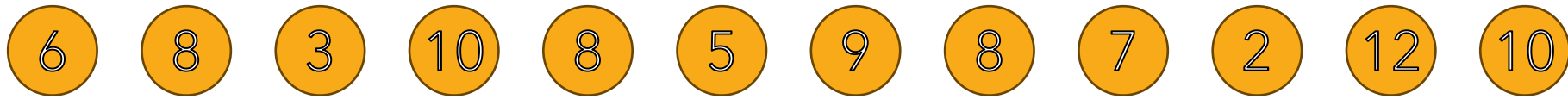
Dynamic programming is a powerful technique precisely because it allows us to use recursive intuition to structure a problem, without needing a prohibitive number of recursive calls to actually construct the solution.

To build a dynamic programming algorithm:

1. Characterize the recursive substructure of the problem.
2. Write a recurrence relation that specifies the optimal value of a larger problem in terms of the optimal values of smaller problems.
3. Compute and record the values (and path to a solution) of every subproblem, in bottom-up fashion, until you reach the target size.
4. Reconstruct the optimal solution from the recorded information.

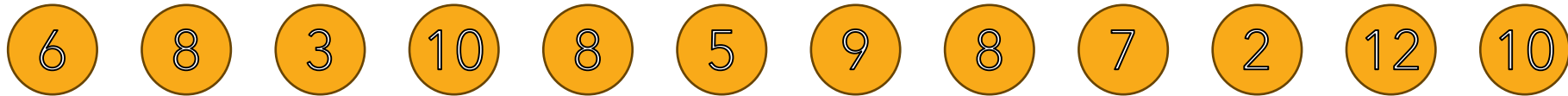
THE COIN-ROW PROBLEM

A game involves a row of n coins, each with a positive value v_i . (The v_i may or may not be distinct.) The goal is to select the set of coins with maximum value, subject to the restriction that two consecutive coins may not be selected.



Dynamic programming is one of the big hammers of algorithm design. Before we bust it out, it's worth asking if a simpler approach might do the job. For this task, what does a brute force algorithm look like? What about greedy?

THE COIN-ROW PROBLEM



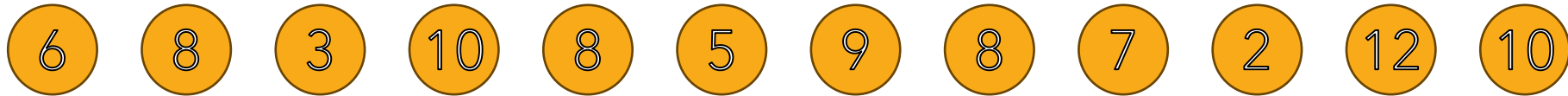
Once we have decided to attempt a dynamic programming solution, we go into our steps.

1. Characterize the recursive substructure of the problem.

How does the solution for n coins depend upon the solution(s) for smaller number(s) of coins?

Often, it is useful to think about one specific element of the larger solution: How does the solution ending with coin c_n depend upon the solutions ending with coins $c_1 \dots c_{n-1}$?

THE COIN-ROW PROBLEM



For selection problems like this one, it can be useful to observe that there are really only two options for coin c_n : use it or don't.

- If we don't use coin c_n , then our best solution will simply be the same as the best solution ending with coin c_{n-1} .
- If we use coin c_n , that will exclude coin c_{n-1} . In that case, we need to know the best solution ending with coin c_{n-2} .

And that's it – those two lines tells us everything we need to know about what the recursive dependencies look like!

QUICK ASIDE: ETYMOLOGY

Why are we calling this dynamic programming? It's really not about writing computer code. Both the term and the technique are due to Richard Bellman, a mathematician who (among other things) worked for the RAND corporation.

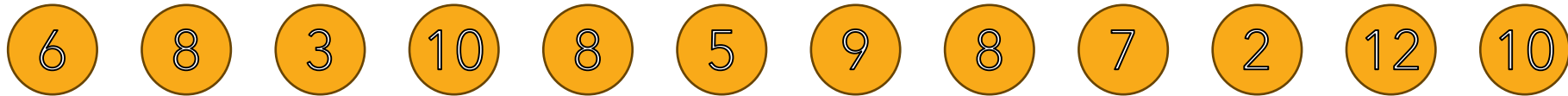
Bellman gave multiple explanations of how he came up with the name, but they share these facts: "program" here is used in the military sense of a system and schedule for making decisions regarding, e.g. logistics and resource allocation. (As in the phrase "Get with the program!") RAND was a major military contractor at the time.

As for "dynamic", in his autobiography Bellman basically said it sounded cool, and it is a term that almost never has a negative connotation.



Image source: Wikipedia

THE COIN-ROW PROBLEM



- If we don't use coin c_n , then our best solution will simply be the same as the best solution ending with coin c_{n-1} .
- If we use coin c_n , that will exclude coin c_{n-1} . In that case, we need to know the best solution ending with coin c_{n-2} .

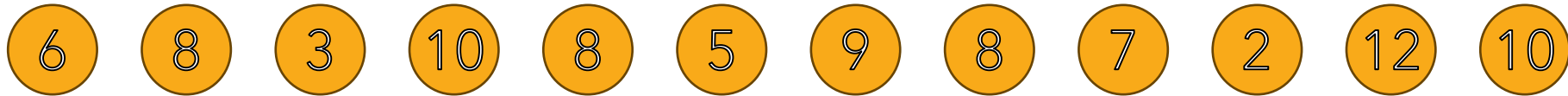
We're on to the next step:

2. Write a recurrence relation that specifies the optimal value of a larger problem in terms of the optimal values of smaller problems.

What is the recurrence relation suggested by the above?

$$OPT(n) = \max(OPT(n-1), c_n + OPT(n-2))$$

THE COIN-ROW PROBLEM



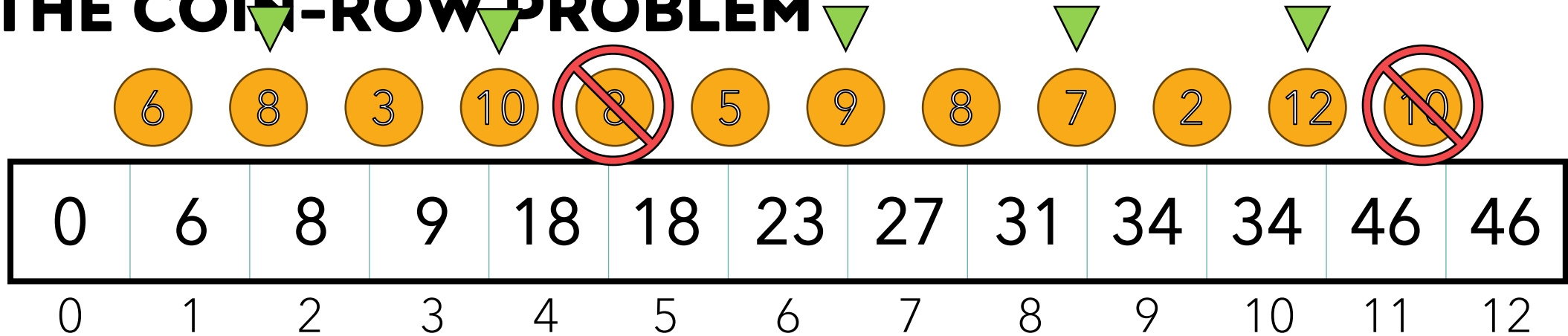
$$OPT(n) = \max(OPT(n-1), c_n + OPT(n-2))$$

$$OPT(0) = 0; OPT(1) = v_1$$

3. Compute and record the values (and path to a solution) of every subproblem, in bottom-up fashion, until you reach the target size.

0	6	8	9	18	18	23	27	31	34	34	46	46
0	1	2	3	4	5	6	7	8	9	10	11	12

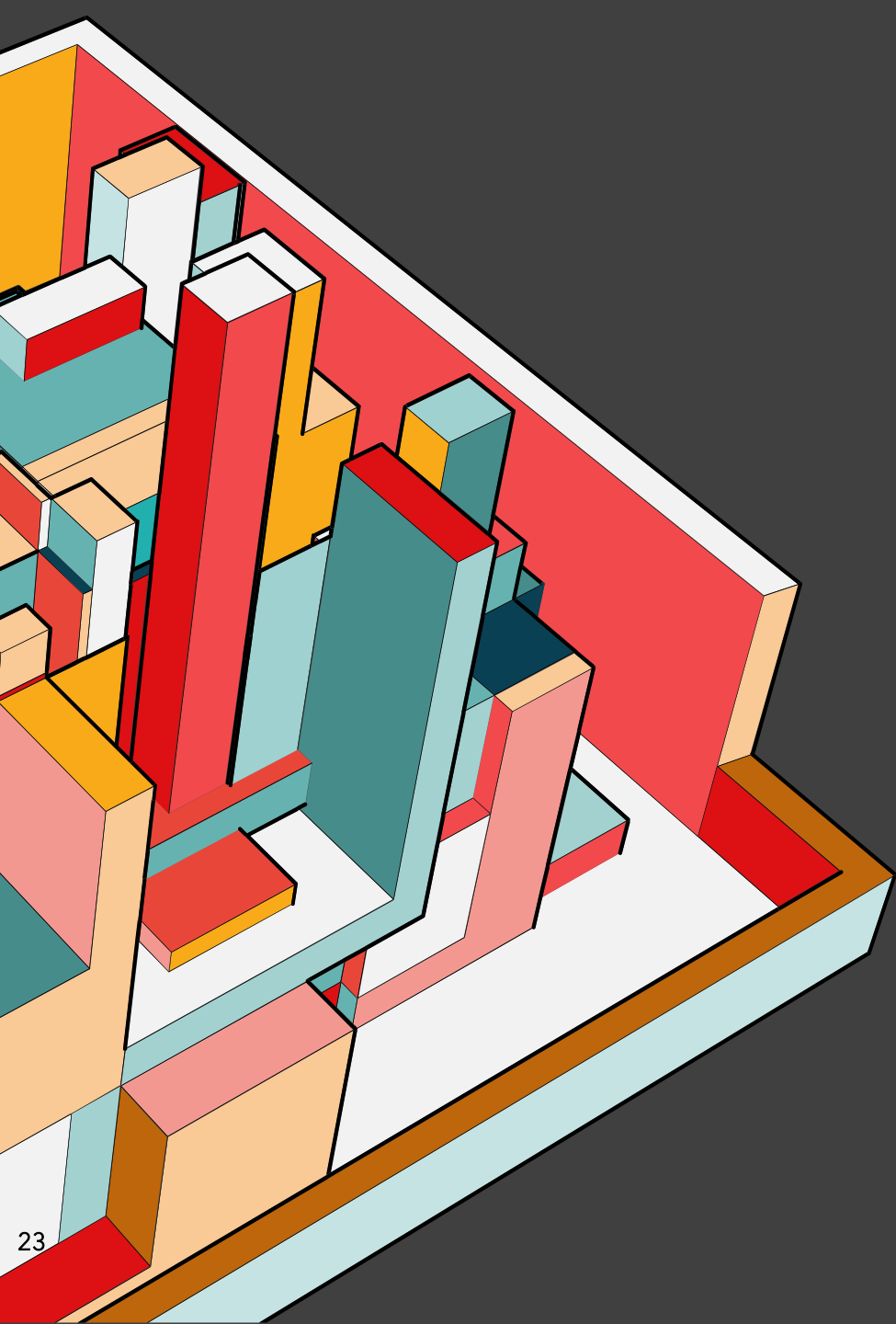
THE COIN-ROW PROBLEM



4. Reconstruct the optimal solution from the recorded information.

In this case, this is pretty straightforward: We start from the end. If $OPT(i) = OPT(i - 1)$, we didn't use c_i . If it doesn't, we DID use c_i . Backsolve from there.

46=46, so we don't use c_{12} and should check c_{11} next. 46≠34 so we used c_{11} and should check c_9 next. Etc.



DANCE BREAK



THE ROD CUTTING PROBLEM

We have an unbroken rod n units long, and a table of prices $p_1 \dots p_n$, where p_i indicates the price of a rod of length i . We can cut the input rod any number of times to form a collection of smaller rods, so long as the smaller rods have integer lengths.

The goal is to find the division of the rod that maximizes the overall price of the collection. (Note that this *could* be to not cut the rod at all.)

As an easy example, let $n = 4$ with the following table of prices:

i	1	2	3	4
p_i	1	5	8	9

SUBMITTY POLL #2

For the rod-cutting problem with $n = 4$ and this table of prices, what is the optimal solution?

i	1	2	3	4
p_i	1	5	8	9

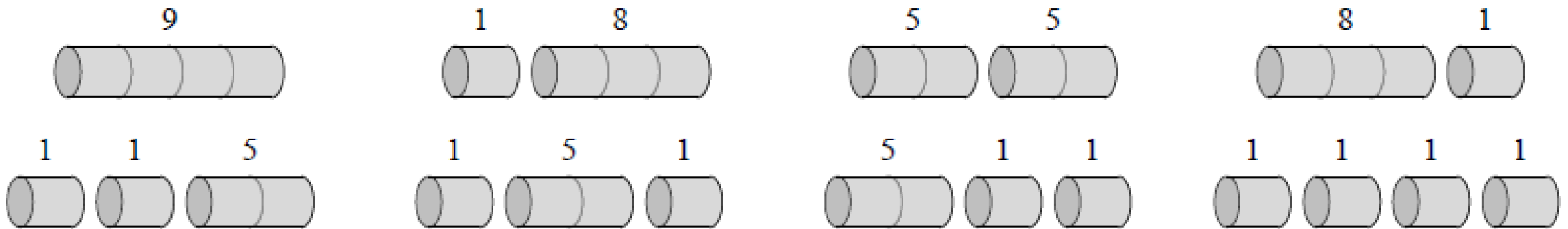


THE ROD CUTTING PROBLEM

i	1	2	3	4
p_i	1	5	8	9

How many ways are there to divide up a rod of length n ?

For length 4, there are 8 possibilities:



If you look at it in this manner, the length n rod has 2^{n-1} possible ways to divide it up.

The poll question had fewer answers because it excluded permutations. The function that counts like this is called the *partition function* and it's wicked complicated. Suffice it to say it's $\Omega(n^k)$ for any value of k .

THE ROD-CUTTING PROBLEM

i	1	2	3	4	5	6	7	8	9	10
p_i	1	5	8	9	10	17	17	20	24	30

Let's scale up to length 10.

We certainly don't want to brute-force by hand now, as there are $2^9 = 512$ possible ways to cut up the rod.

Can we find a recursive substructure to lessen the work?

Well, assuming we make one, let's think about the first cut.

We can produce and sell one piece of any length from 1 to $n - 1$.

Once we have done so, we're left with the task of maximizing the profit of the leftover part.

But how big should that first cut be?

THE ROD-CUTTING PROBLEM

i	1	2	3	4	5	6	7	8	9	10
p_i	1	5	8	9	10	17	17	20	24	30

But how big should that first cut be?

The answer is: We don't know! So we'll have to try everything.

This leads to the following recurrence relation:

$$OPT(n) = \max_{1 \leq i \leq n} (p_i + OPT(n - i)), \text{ where } OPT(0) = 0$$

Again, the idea captured by this recurrence is that we have to sell some piece first; after that, we then maximize our value of the remaining portion (if any).

THE ROD-CUTTING PROBLEM, RECURSIVELY

$$OPT(n) = \max_{1 \leq i \leq n} (p_i + OPT(n - i)), \text{ where } OPT(0) = 0$$

Implementing this directly produces something like this:

```
def cutRod(p,n):  # p is array of prices
    if n==0:  return 0
    result = -∞
    for i = 1 to n:
        result = max(result, p[i] + cutRod(p,n-i))
    return result
```

This, of course, is terrible. It has the same problem as recursive Fibonacci – we are making the same recursive calls over and over and over again. Runtime is $O(2^n)$.

THE ROD-CUTTING PROBLEM, USING DP

$$OPT(n) = \max_{1 \leq i \leq n} (p_i + OPT(n - i)), \text{ where } OPT(0) = 0$$

Let's try it with dynamic programming.

```
def cutRodDP(p,n): # p is array of prices
```

```
    OPT = new array of n+1 elements
```

```
    OPT[0] = 0
```

```
    for j = 1 to n:
```

```
        result = -∞
```

```
        for i = 1 to j:
```

```
            result = max(result, p[i] + OPT[j-i])
```

```
        OPT[j] = result
```

```
    return OPT[n]
```

Two nested
loops: $O(n^2)$
runtime!

LET'S DO IT!

i	1	2	3	4	5	6	7	8	9	10
p_i	1	5	8	9	10	17	17	20	24	30

$$OPT(n) = \max_{1 \leq i \leq n} (p_i + OPT(n - i)), \text{ where } OPT(0) = 0$$

Fill in the remainder of the dynamic programming array now.

0										
0	1	2	3	4	5	6	7	8	9	10

LET'S DO IT!

i	1	2	3	4	5	6	7	8	9	10
p_i	1	5	8	9	10	17	17	20	24	30

$$OPT(n) = \max_{1 \leq i \leq n} (p_i + OPT(n - i)), \text{ where } OPT(0) = 0$$

Fill in the remainder of the dynamic programming array now.

0	1	5	8	10	13	17	18	22	25	30
0	1	2	3	4	5	6	7	8	9	10

$$OPT(3) = \max(8, 5+1, 1+5)$$

$$OPT(4) = \max(9, 8+1, 5+5, 1+8)$$

$$OPT(5) = \max(10, 9+1, 8+5, 5+8, 1+9)$$

RECAP

Dynamic programming is a critically important algorithm design tool.

Its big advantage comes in reusing answers to already solved problems to avoid unnecessary recursive calls.

The four steps of building a dynamic programming algorithm:

1. Characterize the recursive substructure of the problem.
2. Write a recurrence relation that specifies the optimal value of a larger problem in terms of the optimal values of smaller problems.
3. Compute **and record** the values (and path to a solution) of every subproblem, in bottom-up fashion, until you reach the target size.
4. Reconstruct the optimal solution from the recorded information.

BEFORE YOU LEAVE

Please complete this feedback form about today's class.

